

Python Programming — Take-Home Exam (Variant 1)

Handed out: 2026-07-31

Due: 2026-08-14 (23:59) — you have 2 weeks.

Rules

- Work individually. You may use the official Python documentation and the course materials. Plagiarism will be graded with 5.0 (fail).
- Submit one folder (zip or repository link) containing one `.py` file per question (`q1.py`, `q2.py`, `q3.py`, `q4.py`, `q5.py`) plus a **report (PDF, 5–8 pages)**: for each question a description of your approach, the output of your demo, and any problems you ran into; for Question 5 also your real transcript (prompt + raw model reply) and, if attempted, the bonus results. Submitting as a GitHub repository earns **+5 P**.
- Every file must run without errors with Python 3.11+ (`python q1.py`).
- Standard library only — no third-party packages. Exception: Question 5 may use the `openai` package (as in the course).
- Every file needs a short demo block at the bottom showing your functions in action.

Each question is worth 20 points (100 total).

Question 1 — Movie Ratings (Lists and Strings) (20 P)

A cinema collects ratings as one string:

```
data = "Dune:8, Dune:9, Barbie:7, Dune:10, Barbie:9, Oppenheimer:9, Barbie:6"
```

Write a function `parse_ratings(data: str) -> list` that turns this into a list of (title, rating) tuples with the rating as int.

Then write:

1. `average_rating(ratings, title) -> float` — average rating of one movie, rounded to 1 decimal (`round(x, 1)`). Unknown title returns 0.0.
2. `best_movie(ratings) -> str` — title with the highest average.
3. `rating_counts(ratings) -> dict` — how many ratings each movie received.

Extra whitespace around titles or numbers must not break the parser. *Hint:* `data.split(",")` gives the entries, `entry.split(":")` separates title and rating, `.strip()` removes surrounding whitespace — `AI_agent/rag.py` uses `split` the same way.

Question 2 — Gradebook (Dictionaries and Sets) (20 P)

```
grades = {
    "math": {"anna": 1.7, "ben": 2.3, "clara": 1.0},
    "physics": {"ben": 3.0, "clara": 1.3, "david": 2.0},
    "art": {"anna": 1.0, "david": 1.7},
}
```

Write functions:

1. `subjects_of(student)` -> `set` — all subjects a student takes.
2. `takes_all(grades)` -> `set` — students enrolled in *every* subject (use set operations, not counting).
3. `student_average(grades, student)` -> `float` — average grade across the student's subjects, rounded to 2 decimals (`round(x, 2)`). A student who appears nowhere returns 0.0.
4. `honor_roll(grades, limit=1.5)` -> `list` — alphabetically sorted list of students whose average is $\leq$ `limit`.

Question 3 — Recipe Scaler (Functions) (20 P)

Write a function `scale_recipe(name, servings, *ingredients, unit="g", **options)`:

- `ingredients`: arbitrary number of (`ingredient, amount_for_one_serving`) tuples,
- the function prints a shopping list scaled to `servings` (amounts multiplied),
- `unit` is appended to every amount,
- `options`: arbitrary keyword arguments printed as cooking notes (e.g. `oven="180C", time="45min"`),
- if `servings < 1`, the function prints an error line and returns 0,
- the function **returns** a dict `{ingredient: scaled_amount}`.

Show three calls: a simple recipe, one with a different `unit`, one with several `options`. Also show one call with `servings=0`.

Question 4 — Library (Classes and Inheritance) (20 P)

1. Class `Book` with attributes `title, author, year`:
 - `__init__`, `__str__` (nice output like "1984 by George Orwell (1949)"),
 - `__eq__` — equal if title and author match,
 - method `age(current_year)` returning how old the book is.
2. Class `EBook(Book)` with extra attribute `size_mb`:
 - overrides `__str__` (adds the file size),
 - method `download_seconds(mbit_per_s)` returning the download time in seconds (rounded to 1 decimal).
3. Class `Library` that stores books in a list:
 - `add(book)` — ignores the book if an equal one is already there (use your `__eq__`),
 - `find_by_author(author)` -> `list`,
 - `oldest()` -> `Book`,
 - `__len__` so `len(library)` works — implement `__len__(self)` returning the number of stored books.
4. Demo: at least 3 books and 2 ebooks, one duplicate `add` that gets ignored, and every method called once.

Question 5 — Movie Review Extraction Pipeline (LLM + JSON) (20 P)

LLMs return free text — software needs structured data. You will build an extraction pipeline that makes **qwen3-0.6b** produce valid JSON and defends itself when the model doesn't.

Setup: LM Studio running locally with **qwen3-0.6b** loaded, server at `http://localhost:1234/v1` — exactly like `AI_agent/rag.py` from the course. This question may use the `openai` package (`OpenAI(base_url="http://localhost:1234/v1", api_key="lm-studio")`). Always use `temperature=0` (except in the bonus). Read only `message.content` — LM Studio puts the model's thinking into a separate `reasoning_content` field; ignore it.

Build, in this order:

1. `ask_llm(prompt: str, system: str) -> str` — sends one chat completion (system + user message) and returns the reply text.
2. `build_prompt(text: str) -> str` and `extract_review(text: str) -> dict` — the prompt instructs the model to reply with **only** a JSON object

```
{"title": "Dune", "rating": 9, "sentiment": "positive"}
```

(`title`: string, `rating`: integer 1–10, `sentiment`: "positive" or "negative") for one messy movie review.

Model replies are not always valid JSON. Copy this ready-made helper into your file — it returns the parsed object, or `None` if the text is not valid JSON (you do not need to understand its inside):

```
def try_parse_json(text):
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        return None
```

`extract_review` parses the reply with `try_parse_json`.

3. `check_review(d) -> str` — returns an error message (precise reason) if: `d` is `None`, keys are not exactly these 3, `title` is not a non-empty string, `rating` is not an integer in 1–10 (a `bool` does not count!), or `sentiment` is neither "positive" nor "negative". Returns the empty string "" if everything is valid.
4. `extract_with_retry(text: str, max_attempts=3) -> dict` — the defense: call the model, parse with `try_parse_json`, check with `check_review`; if the error message is not empty, build a **new prompt that quotes the error message** and demands corrected JSON, then try again. After `max_attempts` failures print an error line (which review failed and why) and return `None`.
5. The pipeline: a list `REVIEWS` of at least 5 messy reviews in different styles, e.g. "Saw Dune yesterday - absolutely loved it, easily 9/10!", "Barbie was ok I guess. 6 out of 10", one that writes the rating in words ("gave it a seven"), one clearly negative. Run all of them through `extract_with_retry`, skip `None` results, and print a small result table plus a count of positive vs. negative reviews.

Demo: the demo block at the bottom runs the whole pipeline against LM Studio and additionally contains **one real transcript** (your prompt + the raw model reply) pasted as a comment.

Bonus (+5 P): measure instead of trusting: run the 5 reviews once with `temperature=0` and once with `temperature=1.5`; count for each run how many reviews needed retries or failed entirely. 3–5 line comment on what you observe.